

# SQL Technical Interview Mastery Guide

10 Verified Practice Questions, Query Solutions, and Execution Benchmarks

Target Database: `test-company.db`

September 6, 2026

## Relational Database Schema Overview

This technical preparation document is grounded in the live relational database `test-company.db`, structured across the following schema entities:

- **employees** (2,500 records): Personal details, salary, hire date, status, department ID, pay type ID, salary grade ID.
- **departments** (10 records): Department names and unique primary keys.
- **pay\_types** (2 records): Hourly vs. Salary compensation schemes.
- **salary\_grades** (5 records): Grade tiers and management levels.
- **customers** (100 records): Enterprise client companies and their assigned employee representative.
- **transactions** (5,000 records): Sales amounts, service types, transaction types, and sales channels.

All queries in this document have been pre-executed directly against the database engine to guarantee absolute accuracy and provide verified reference benchmarks.

## Contents

|                                                                                          |          |
|------------------------------------------------------------------------------------------|----------|
| <b>1 SQL Technical Interview Questions &amp; Benchmarks</b>                              | <b>2</b> |
| Question 1: Basic Aggregation Filtering: Active Employee Count and Average Salary . .    | 2        |
| Question 2: Aggregated Filtering: High-Headcount Departments via HAVING . . . . .        | 2        |
| Question 3: Subqueries CTEs: Employees Earning Above Department Average . . . . .        | 3        |
| Question 4: Window Functions: Top 2 Highest-Paid Employees per Department . . . . .      | 4        |
| Question 5: Conditional Aggregation: Pivoting Sales Channels by Service Type . . . . .   | 5        |
| Question 6: Multi-Table Relational Joins: Top Sales Representatives by Revenue . . . . . | 6        |
| Question 7: LEFT JOIN Status Breakdown: Salary Grade Distributions . . . . .             | 6        |
| Question 8: Date Handling Cohort Retention Rate Analysis . . . . .                       | 7        |
| Question 9: Cumulative Window Functions: Running Balance of Transactions . . . . .       | 8        |
| Question 10: Customer Segmentation Quartile Analysis via NTILE() . . . . .               | 9        |

# 1 SQL Technical Interview Questions & Benchmarks

## Question 1: Basic Aggregation Filtering: Active Employee Count and Average Salary

**Tested SQL Competencies:** Basic Aggregations (COUNT, AVG, ROUND), WHERE filter, JOIN, GROUP BY

### Problem Statement:

Write an SQL query to retrieve the total count of currently **Active** employees and their average salary (rounded to 2 decimal places) grouped by their pay type (e.g., Hourly vs Salary). Sort the result by average salary in descending order.

### Official SQL Solution:

```
SELECT
    pt.description AS pay_type,
    COUNT(e.id) AS active_employee_count,
    ROUND(AVG(e.salary), 2) AS avg_salary
FROM employees e
JOIN pay_types pt ON e.pay_type_id = pt.id
WHERE e.status = 'Active'
GROUP BY pt.description
ORDER BY avg_salary DESC;
```

### Verified Execution Benchmark:

| pay_type | active_employee_count | avg_salary |
|----------|-----------------------|------------|
| Salary   | 619                   | 115371.58  |
| Hourly   | 624                   | 112674.99  |

### Interview Key Takeaways:

This question evaluates candidate proficiency with fundamental multi-table joins (`employees` ↔ `pay_types`), row filtering prior to grouping via `WHERE`, and numeric aggregation rounding.

## Question 2: Aggregated Filtering: High-Headcount Departments via HAVING

**Tested SQL Competencies:** GROUP BY, HAVING clause vs. WHERE clause, Multi-column Aggregations (MIN, MAX)

### Problem Statement:

Find all company departments that have **at least 125 active employees**. For each qualifying department, output the department name, the number of active employees, the lowest salary, and the highest salary in that department. Sort descending by employee count.

### Official SQL Solution:

```
SELECT
    d.department_name,
    COUNT(e.id) AS active_employee_count,
    MIN(e.salary) AS min_salary,
    MAX(e.salary) AS max_salary
FROM departments d
JOIN employees e ON d.id = e.department_id
HAVING COUNT(e.id) >= 125
ORDER BY active_employee_count DESC;
```

```

WHERE e.status = 'Active'
GROUP BY d.department_name
HAVING COUNT(e.id) >= 125
ORDER BY active_employee_count DESC;

```

#### Verified Execution Benchmark:

| department_name  | active_employee_count | min_salary | max_salary |
|------------------|-----------------------|------------|------------|
| Marketing        | 136                   | 31223      | 199411     |
| Finance          | 132                   | 31476      | 198971     |
| Executive        | 131                   | 30189      | 198537     |
| Communications   | 131                   | 30002      | 199563     |
| Customer Service | 125                   | 30002      | 199297     |

#### Interview Key Takeaways:

Tests understanding of the SQL order of execution. Candidates must distinguish between filtering rows before aggregation (`WHERE status = 'Active'`) versus filtering groups after aggregation (`HAVING COUNT(e.id) >= 125`).

### Question 3: Subqueries CTEs: Employees Earning Above Department Average

**Tested SQL Competencies:** Common Table Expressions (WITH), Two-stage Aggregations, Conditional Join Matching

#### Problem Statement:

Identify active employees whose salary exceeds the average salary of their specific department. Return the employee name, department name, employee salary, and the department average salary (rounded to 2 decimals). Order by employee salary descending and limit output to the top 5.

#### Official SQL Solution:

```

WITH DeptAvg AS (
    SELECT
        department_id,
        AVG(salary) AS avg_dept_salary
    FROM employees
    WHERE status = 'Active'
    GROUP BY department_id
)
SELECT
    e.name AS employee_name,
    d.department_name,
    e.salary,
    ROUND(da.avg_dept_salary, 2) AS dept_avg_salary
FROM employees e
JOIN departments d ON e.department_id = d.id
JOIN DeptAvg da ON e.department_id = da.department_id
WHERE e.status = 'Active'
    AND e.salary > da.avg_dept_salary
ORDER BY e.salary DESC
LIMIT 5;

```

**Verified Execution Benchmark:**

| employee_name | department_name        | salary | dept_avg_salary |
|---------------|------------------------|--------|-----------------|
| Sofia Roberts | Information Technology | 199690 | 113535.64       |
| Evelyn Lopez  | Information Technology | 199600 | 113535.64       |
| Ava Adams     | Communications         | 199563 | 108471.4        |
| Sofia Jackson | Marketing              | 199411 | 112819.71       |
| Nora Baker    | Customer Service       | 199297 | 115006.68       |

**Interview Key Takeaways:**

Common technical interview question to test whether the candidate can isolate departmental metrics using a CTE or correlated subquery before comparing individual row-level figures against group-level benchmarks.

**Question 4: Window Functions: Top 2 Highest-Paid Employees per Department**

**Tested SQL Competencies:** Window Functions (`DENSE_RANK()`), `PARTITION BY`, `ORDER BY` inside window, CTE filtering

**Problem Statement:**

For each department, find the top 2 highest-paid active employees using the `DENSE_RANK()` window function. Return the department name, employee name, salary, and their departmental rank. Display the first 10 rows sorted alphabetically by department and rank.

**Official SQL Solution:**

```
WITH RankedEmployees AS (
    SELECT
        d.department_name,
        e.name AS employee_name,
        e.salary,
        DENSE_RANK() OVER (
            PARTITION BY e.department_id
            ORDER BY e.salary DESC
        ) AS salary_rank
    FROM employees e
    JOIN departments d ON e.department_id = d.id
    WHERE e.status = 'Active'
)
SELECT
    department_name,
    employee_name,
    salary,
    salary_rank
FROM RankedEmployees
WHERE salary_rank <= 2
ORDER BY department_name ASC, salary_rank ASC
LIMIT 10;
```

**Verified Execution Benchmark:**

| department_name        | employee_name | salary | salary_rank |
|------------------------|---------------|--------|-------------|
| Communications         | Ava Adams     | 199563 | 1           |
| Communications         | Oliver Lee    | 199077 | 2           |
| Customer Service       | Nora Baker    | 199297 | 1           |
| Customer Service       | Leo Adams     | 197888 | 2           |
| Executive              | Ava Lee       | 198537 | 1           |
| Executive              | John Jackson  | 197811 | 2           |
| Finance                | Levi Scott    | 198971 | 1           |
| Finance                | Luca Wilson   | 197295 | 2           |
| Information Technology | Sofia Roberts | 199690 | 1           |
| Information Technology | Evelyn Lopez  | 199600 | 2           |

### Interview Key Takeaways:

Window functions are essential in mid-to-senior SQL interviews. The candidate must know that window functions cannot be filtered directly in a `WHERE` clause without wrapping in a CTE or subquery, and understand the difference between `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()`.

## Question 5: Conditional Aggregation: Pivoting Sales Channels by Service Type

**Tested SQL Competencies:** Conditional Aggregation (`CASE WHEN` inside `SUM`), Cross-tabulation / Pivoting

### Problem Statement:

For each `service_type` in transactions, calculate the total revenue generated through 'Online' sales, 'Phone' sales, and the combined overall revenue. Round all monetary values to 2 decimal places and sort by total revenue descending.

### Official SQL Solution:

```
SELECT
    service_type,
    ROUND(SUM(CASE WHEN sales_type = 'Online' THEN net_amount ELSE 0 END), 2) AS online_revenue,
    ROUND(SUM(CASE WHEN sales_type = 'Phone' THEN net_amount ELSE 0 END), 2) AS phone_revenue,
    ROUND(SUM(net_amount), 2) AS total_revenue
FROM transactions
GROUP BY service_type
ORDER BY total_revenue DESC;
```

### Verified Execution Benchmark:

| service_type | online_revenue | phone_revenue | total_revenue |
|--------------|----------------|---------------|---------------|
| Service      | 2711019.43     | 2403741.7     | 5114761.13    |
| Maintenance  | 2348240.26     | 2672761.31    | 5021001.57    |
| Consulting   | 2461679.51     | 2474814.49    | 4936494.0     |
| One-time     | 2270628.11     | 2589030.31    | 4859658.42    |
| Contract     | 2302487.79     | 2444295.47    | 4746783.26    |

### Interview Key Takeaways:

Tests data transformation and pivoting techniques without requiring proprietary pivot syntax. Conditional aggregation with `SUM(CASE WHEN ...)` is a cornerstone of business intelligence and analytics SQL.

### Question 6: Multi-Table Relational Joins: Top Sales Representatives by Revenue

**Tested SQL Competencies:** 4-Table Inner Joins (employees, departments, customers, transactions), COUNT(DISTINCT)

**Problem Statement:**

Find the top 5 employee account representatives whose assigned customers generated the highest total transaction revenue. Display representative name, department name, number of distinct customers managed, and total revenue generated.

**Official SQL Solution:**

```
SELECT
    e.name AS representative_name,
    d.department_name,
    COUNT(DISTINCT c.id) AS managed_customers_count,
    ROUND(SUM(t.net_amount), 2) AS total_revenue_generated
FROM employees e
JOIN departments d ON e.department_id = d.id
JOIN customers c ON e.id = c.representative_id
JOIN transactions t ON c.id = t.customer_id
GROUP BY e.id, e.name, d.department_name
ORDER BY total_revenue_generated DESC
LIMIT 5;
```

**Verified Execution Benchmark:**

| representative_name | department_name  | managed_customers_count | total_revenue_generated |
|---------------------|------------------|-------------------------|-------------------------|
| Henry Hill          | Customer Service | 1                       | 288351.34               |
| Harper Davis        | Risk Management  | 1                       | 285654.47               |
| Violet Taylor       | Purchasing       | 1                       | 285309.34               |
| Charlotte Martin    | Payroll          | 1                       | 284945.14               |
| Noah Williams       | Purchasing       | 1                       | 284196.04               |

**Interview Key Takeaways:**

Demonstrates ability to navigate complex entity-relationship graphs across 4 tables while correctly avoiding duplicate count inflation using COUNT(DISTINCT c.id).

### Question 7: LEFT JOIN Status Breakdown: Salary Grade Distributions

**Tested SQL Competencies:** LEFT JOIN, Handling Non-Matching Outer Data, Pivot Aggregations

**Problem Statement:**

List all salary grades (ID, tier description, and full title) along with the count of active employees, terminated employees, and total employees assigned to each grade. Ensure all salary grades are preserved in the result even if an employee count is 0.

**Official SQL Solution:**

```

SELECT
    sg.id AS grade_id,
    sg.description AS grade_tier,
    sg.long_text AS grade_title,
    COUNT(CASE WHEN e.status = 'Active' THEN 1 END) AS active_count,
    COUNT(CASE WHEN e.status = 'Terminated' THEN 1 END) AS terminated_count,
    COUNT(e.id) AS total_employees
FROM salary_grades sg
LEFT JOIN employees e ON sg.id = e.salary_grade_id
GROUP BY sg.id, sg.description, sg.long_text
ORDER BY sg.id ASC;

```

#### Verified Execution Benchmark:

| grade_id | grade_tier | grade_title       | active_count | terminated_count | total_employees |
|----------|------------|-------------------|--------------|------------------|-----------------|
| 1        | Tier 1     | Worker            | 375          | 350              | 725             |
| 2        | Tier 2     | Lower Management  | 138          | 154              | 292             |
| 3        | Tier 3     | Middle Management | 162          | 156              | 318             |
| 4        | Tier 4     | Upper Management  | 158          | 168              | 326             |
| 5        | Tier 5     | Senior Management | 410          | 429              | 839             |

#### Interview Key Takeaways:

Validates grasp of LEFT JOIN to preserve dimension rows when matching fact tables, along with selective status counting.

### Question 8: Date Handling Cohort Retention Rate Analysis

**Tested SQL Competencies:** Date pattern parsing, Categorical Cohorting, Percentage Metrics, Integer-to-Float Division

#### Problem Statement:

Group employees into hiring cohorts by decade (1980s, 1990s, 2000s, 2010s, 2020s). Calculate total hired, count of currently active staff, and the active retention rate percentage ( $\text{Active} / \text{Total} * 100$ , rounded to 2 decimal places).

#### Official SQL Solution:

```

SELECT
    CASE
        WHEN hire_date LIKE '198%' THEN '1980s'
        WHEN hire_date LIKE '199%' THEN '1990s'
        WHEN hire_date LIKE '200%' THEN '2000s'
        WHEN hire_date LIKE '201%' THEN '2010s'
        WHEN hire_date LIKE '202%' THEN '2020s'
        ELSE 'Other'
    END AS hire_decade,
    COUNT(*) AS total_hired,
    SUM(CASE WHEN status = 'Active' THEN 1 ELSE 0 END) AS active_employees,
    ROUND(100.0 * SUM(CASE WHEN status = 'Active' THEN 1 ELSE 0 END) / COUNT(*), 2) AS retention_rate
FROM employees
GROUP BY hire_decade

```

```
HAVING hire_decade != 'Other'
ORDER BY hire_decade ASC;
```

**Verified Execution Benchmark:**

| hire_decade | total_hired | active_employees | retention_rate_pct |
|-------------|-------------|------------------|--------------------|
| 1980s       | 87          | 46               | 52.87              |
| 1990s       | 248         | 114              | 45.97              |
| 2000s       | 435         | 218              | 50.11              |
| 2010s       | 809         | 402              | 49.69              |
| 2020s       | 920         | 463              | 50.33              |

**Interview Key Takeaways:**

Assesses date manipulation, cohort grouping, and avoiding integer truncation division errors (e.g. multiplying by 100.0).

**Question 9: Cumulative Window Functions: Running Balance of Transactions**

**Tested SQL Competencies:** Cumulative Window Framing (ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), Window SUM()

**Problem Statement:**

For Customer ID = 1 (NVIDIA), list their first 6 transactions ordered by transaction ID. Display transaction ID, service type, transaction type, sales type, transaction net amount, and the running cumulative total amount spent up to that transaction.

**Official SQL Solution:**

```
SELECT
    id AS transaction_id,
    service_type,
    transaction_type,
    sales_type,
    net_amount,
    ROUND(SUM(net_amount) OVER (
        ORDER BY id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ), 2) AS running_total_amount
FROM transactions
WHERE customer_id = 1
ORDER BY id ASC
LIMIT 6;
```

**Verified Execution Benchmark:**

| transaction_id | service_type | transaction_type | sales_type | net_amount | running_total_amount |
|----------------|--------------|------------------|------------|------------|----------------------|
| 37             | One-time     | Buy              | Phone      | 9182.39    | 9182.39              |
| 88             | Consulting   | Transfer         | Online     | 1800.85    | 10983.24             |
| 182            | Contract     | Sale             | Online     | 7186.42    | 18169.66             |
| 231            | One-time     | Transfer         | Phone      | 6700.46    | 24870.12             |
| 285            | One-time     | Buy              | Phone      | 9429.71    | 34299.83             |
| 305            | Service      | Transfer         | Phone      | 4283.43    | 38583.26             |

**Interview Key Takeaways:**

Evaluates knowledge of running sums, ledger calculations, and explicit window frame specifications (ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

**Question 10: Customer Segmentation Quartile Analysis via NTILE()**

**Tested SQL Competencies:** Statistical Ranking (NTILE(4)), Customer Lifetime Value Aggregation, CTE

**Problem Statement:**

Divide all 100 enterprise customers into 4 spending quartiles (where Quartile 1 contains the top 25% highest spenders) based on their total transaction volume. Display the top 5 highest-spending companies in Quartile 1.

**Official SQL Solution:**

```
WITH CustomerSpending AS (
    SELECT
        c.id AS customer_id,
        c.company_name,
        ROUND(SUM(t.net_amount), 2) AS total_spend,
        NTILE(4) OVER (ORDER BY SUM(t.net_amount) DESC) AS spending_quartile
    FROM customers c
    JOIN transactions t ON c.id = t.customer_id
    GROUP BY c.id, c.company_name
)
SELECT
    customer_id,
    company_name,
    total_spend,
    spending_quartile
FROM CustomerSpending
WHERE spending_quartile = 1
ORDER BY total_spend DESC
LIMIT 5;
```

**Verified Execution Benchmark:**

| customer_id | company_name | total_spend | spending_quartile |
|-------------|--------------|-------------|-------------------|
| 68          | AppLovin     | 288351.34   | 1                 |
| 4           | Microsoft    | 285654.47   | 1                 |
| 51          | Verizon      | 285309.34   | 1                 |
| 79          | Honeywell    | 284945.14   | 1                 |
| 30          | Coca-Cola    | 284196.04   | 1                 |

**Interview Key Takeaways:**

Tests knowledge of statistical distribution functions in SQL like NTILE(n), widely utilized in customer segmentation and RFM (Recency, Frequency, Monetary) analytics.